

```
In [ ]: # Importation des bibliothèques nécessaires pour le traitement de texte
from textblob import TextBlob
import nltk

# Téléchargement des ressources NLTK nécessaires
nltk.download('punkt')
nltk.download('stopwords') # Pour supprimer les mots courants
nltk.download('averaged_perceptron_tagger') # Pour étiqueter les parties
nltk.download('wordnet') # Pour la lemmatisation
! python -m textblob.download_corpora

# Exemples d'avis avec sentiment associé (tels que fournis)
```

```
[nltk_data] Downloading package punkt to /Users/mathis/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] /Users/mathis/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /Users/mathis/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package wordnet to /Users/mathis/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
zsh:1: command not found: python
```

```
In [ ]: # Importation des bibliothèques de manipulation de données et de visualis
import pandas as pd
import re
from collections import Counter

import matplotlib.pyplot as plt
import seaborn as sns
import nltk
from textblob import TextBlob, Word
from wordcloud import WordCloud
from nltk.corpus import stopwords

# Chargement du jeu de données Amazon
avis_df = pd.read_csv("../datasets/amazon.csv")
avis_df
```

Out []:

	reviewText	Positive
0	This is a one of the best apps acording to a b...	1
1	This is a pretty good version of the game for ...	1
2	this is a really cool game. there are a bunch ...	1
3	This is a silly game and can be frustrating, b...	1
4	This is a terrific game on any pad. Hrs of fun...	1
...	...	...
19995	this app is fricken stupid.it froze on the kin...	0
19996	Please add me!!!! I need neighbors! Ginger101...	1
19997	love it! this game. is awesome. wish it had m...	1
19998	I love love love this app on my side of fashio...	1
19999	This game is a rip off. Here is a list of thin...	0

20000 rows × 2 columns

1. Can you perform a sentiment analysis on the collection of product reviews, classify each one as 'positive', 'negative', or 'neutral', and then generate a bar chart to visualize the distribution of these sentiments?

```
In [ ]: # Identification de la colonne de texte dans le dataset
colonne_texte = 'reviewText' if 'reviewText' in avis_df.columns else avis
textes_avis = avis_df[colonne_texte].fillna('').astype(str)

# Filtre pour ne garder que les textes non vides
textes_avis = textes_avis[textes_avis.str.strip() != '']

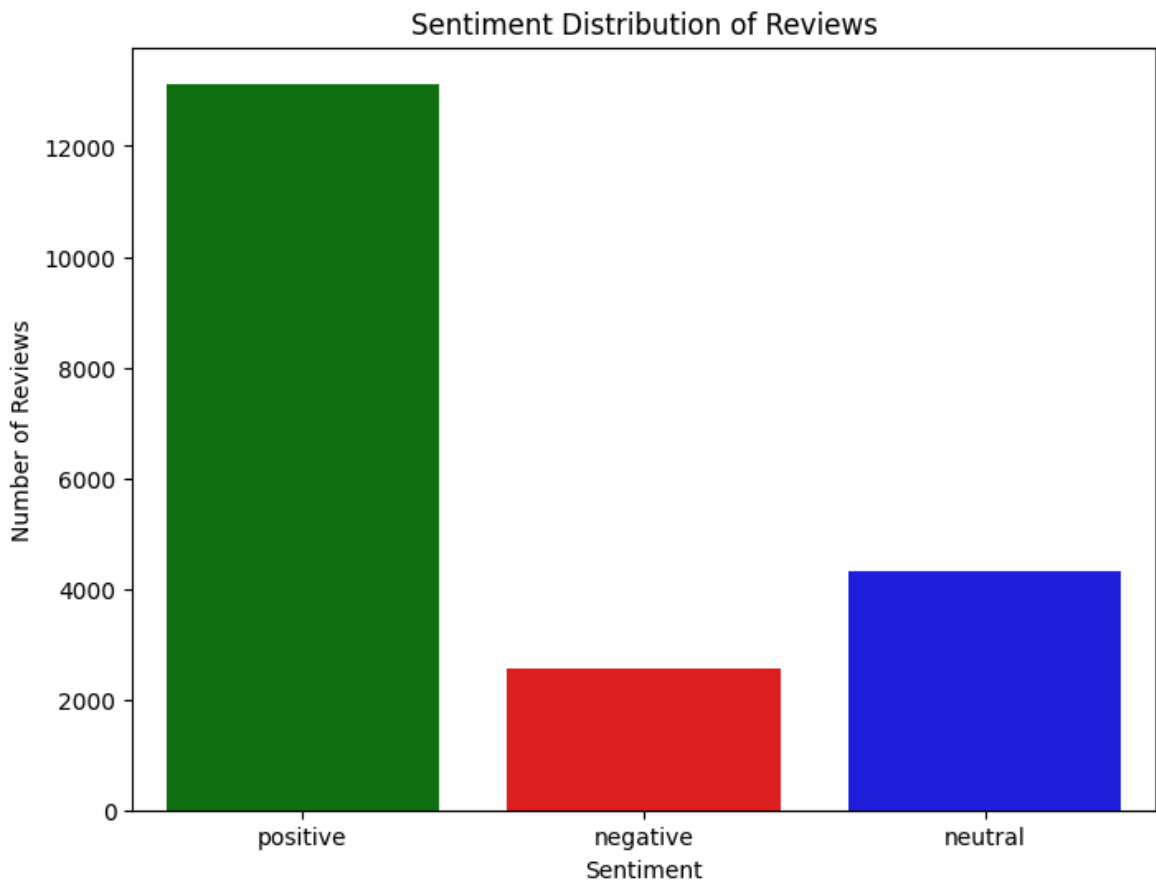
# Calcul de la polarité pour chaque avis
polarites = textes_avis.apply(lambda x: TextBlob(x).sentiment.polarity)

# Classification des sentiments selon la polarité
sentiments_classes = polarites.apply(lambda p: 'positive' if p > 0.1 else

# Comptage du nombre d'avis par catégorie de sentiment
compteur_sentiments = Counter(sentiments_classes)

# Création du graphique en barres de la distribution des sentiments
plt.figure(figsize=(8, 6))
sns.barplot(
    x=list(compteur_sentiments.keys()),
    y=list(compteur_sentiments.values()),
    hue=list(compteur_sentiments.keys()),
    palette=['green', 'red', 'blue'],
    legend=False
)
plt.title('Distribution des sentiments des avis')
plt.xlabel('Sentiment')
plt.ylabel('Nombre d\'avis')
plt.show()
```

```
# Affichage du bilan textuel
print('Colonne de texte utilisée:', colonne_texte)
print('Avis analysés:', len(textes_avis))
print('Compte des sentiments:', compteur_sentiments)
```



Text column used: reviewText

Reviews analyzed: 20000

Sentiment counts: Counter({'positive': 13127, 'neutral': 4324, 'negative': 2549})

- convert the text to lowercase, remove punctuation and numbers, filter out common stopwords, and then lemmatize the remaining words using their appropriate part-of-speech tags.

```
In [ ]: # Téléchargement silencieux des compléments NLTK manquants
nltk.download('punkt', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)
nltk.download('wordnet', quiet=True)

# Liste des mots vides (stopwords) en anglais à filtrer
mots_vides = set(stopwords.words('english'))

# Fonction pour adapter les tags au format de WordNet
def obtenir_pos_wordnet(tag):
    if tag.startswith('J'):
        return 'a' # Adjectifs
    if tag.startswith('V'):
        return 'v' # Verbes
    if tag.startswith('N'):
        return 'n' # Noms
    if tag.startswith('R'):
```

```

        return 'r' # Adverbes
    return 'n'      # Noms par défaut

tous_lemmes = []
tous_tags_pos = []

# Traitement, nettoyage et lemmatisation pour chaque texte
for texte in textes_avis:
    # On enlève tout ce qui n'est pas une lettre et on passe en minuscule
    texte_propre = re.sub(r'^a-zA-Z\s', ' ', texte.lower())
    texte_propre = re.sub(r'\s+', ' ', texte_propre).strip()

    if texte_propre == '':
        continue

    # On parcourt chaque mot tagué pour le vérifier et le lemmatiser
    for mot, tag in TextBlob(texte_propre).tags:
        if mot in mots_vides or len(mot) <= 1:
            continue

        # Extraction du lemme
        lemme = Word(mot).lemmatize(obtenir_pos_wordnet(tag))
        tous_lemmes.append(lemme)
        tous_tags_pos.append(tag)

print('Nombre total de tokens lemmatisés:', len(tous_lemmes))

```

Number of lemmatized tokens: 328184

3. after the text has been fully preprocessed and lemmatized, could you calculate the frequency of each unique word and then display a bar graph showing the top 15 most common words?

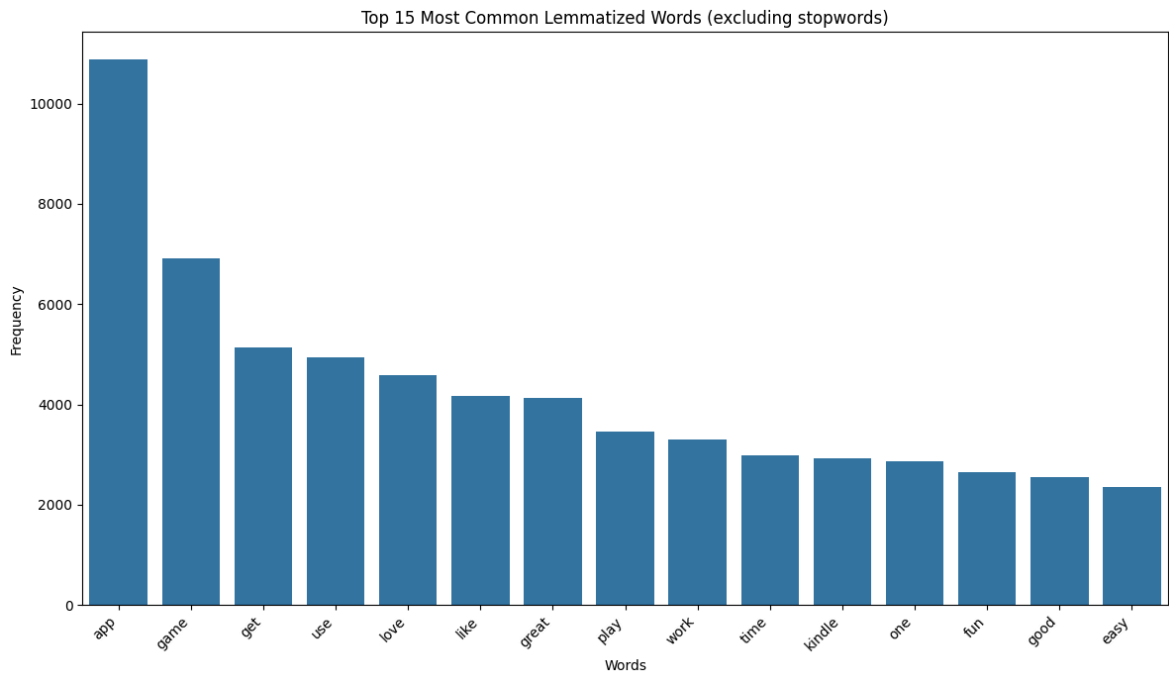
```

In [ ]: # Calcul des fréquences de mots lemmatisés
frequency_mots = Counter(tous_lemmes)
top_15_mots = frequency_mots.most_common(15)

if len(top_15_mots) > 0:
    mots, comptes = zip(*top_15_mots)

    # Affichage du diagramme en barres des mots les plus fréquents
    plt.figure(figsize=(12, 7))
    sns.barplot(x=list(mots), y=list(comptes))
    plt.title('Top 15 des mots lemmatisés les plus fréquents (sans stopwo
    plt.xlabel('Mots')
    plt.ylabel('Fréquence')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
else:
    print('Aucun mot trouvé après le prétraitement.')

```



4. generate a word cloud based on the entire corpus of lemmatized review words to provide a visual summary of the most prominent terms.

```
In [ ]: # Création du corpus complet (tous les lemmes attachés par un espace)
corpus_complet = ' '.join(tous_lemmes)

if corpus_complet.strip() != '':
    # Génération du Nuage de Mots
    nuage_mots = WordCloud(width=800, height=400, background_color='white')

    plt.figure(figsize=(10, 8))
    plt.imshow(nuage_mots, interpolation='bilinear')
    plt.axis('off')
    plt.title('Résumé visuel des avis (Nuage de mots)')
    plt.show()
else:
    print('Aucun mot disponible pour le nuage de mots.')
```



5. Visualize the frequency of nouns, verbs, adjectives in this text.

```
In [ ]: # Initialisation du dictionnaire de comptage des catégories grammaticales
comptes_pos = Counter({'noms': 0, 'verbes': 0, 'adjectifs': 0})

# Décompte en fonction de la première lettre du Tag POS
for tag in tous_tags_pos:
    if tag.startswith('N'):
        comptes_pos['noms'] += 1
    elif tag.startswith('V'):
        comptes_pos['verbes'] += 1
    elif tag.startswith('J'):
        comptes_pos['adjectifs'] += 1

# Affichage des fréquences de types de mots
plt.figure(figsize=(8, 6))
sns.barplot(
    x=['Noms', 'Verbes', 'Adjectifs'],
    y=[comptes_pos['noms'], comptes_pos['verbes'], comptes_pos['adjectifs']]
)

plt.title('Fréquence des Noms, Verbes et Adjectifs')
plt.xlabel('Catégorie POS (Part-of-Speech)')
plt.ylabel('Nombre d'occurrences')
plt.show()
```

